1.

One of my roommates, Jonathan, loves glizzies. Unfortunately, far too often, he misplaces his glizzies somewhere in the apartment. Now, the glizzy gobbler managed to lose his glizzies again, and he needs to find them. Now, he only needs one pack of glizzies, but somehow he's managed to lose his entire supply of three glizzies! Further, he seems to have lost his tongs! He needs to find all of his glizzies, and then one of his pairs of tongs afterward (after all, he can't pick up the glizzies if he has tongs in his hands)! They seem to have been lost in some especially dark corner of the apartment this time. Jonathan cannot find them on his own using his antediluvian searching techniques, so we must help Jonathan find them by communicating a traditional searching algorithm to him by yelling at him from across the apartment. Hopefully, Jonny will be able to prepare his glizzies with our help.

2.

a. States are defined as various locations Jonathan can be in within the apartment, illustrated as a square grid composed of locations where he can stand and edges that he cannot pass through.
b. The initial state is where Jonny starts in the search algorithm—in his bedroom. At this time, he will have no glizzies and no tongs in his possession.
c. The only actions Jonny can make are moving into an adjacent square not blocked off by an edge. The successor function will change the state into one where Jonny has moved to the chosen valid location.
d. Has Jonny found all three glizzies? Did he just interact with the tongs after finding all three glizzies? Both need to be true concurrently.
e. The cost function will be how many steps it takes for him to optimally find the glizzies and tongs (so how many tiles are traversed in the search functions found path—only counts new tiles).
f. Grid restrictions: he has to follow the layout of the apartment (cannot walk through walls).

The program prioritizes finding all three glizzies before it finds one of the goals. It then finds the path that does this optimally. This is all kind of done in the same step, but it all affects the cost function and the optimal cost found.

Testing

All directional path costs are 1 for the sake of testing (kind of an unnecessary modification cause we desire 1 for everything, but whatever)

Test Mutation (BFS)	Number of expanded states	Path length	Total Path Cost
Start State: 32 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49	220	14	13

Start State: 179 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49	120	22	21
Start State: 143 Hot Dogs: 62, 47, 82 Goals: 144, 77, 32	122	14	13

Test Mutation (UCS)	Number of expanded states	Path length	Total Path Cost
Start State: 32 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49	220	14	13
Start State: 179 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49	120	22	21
Start State: 143 Hot Dogs: 62, 47, 82 Goals: 144, 77, 32	122	14	13

Breadth First Search seemed to branch out more and be more spread out across the entire environment. One of the most noticeable differences between the performance of the two algorithms was in the first test, where UCS missed the last glizzy (right next to the start) and went on a wild goose chase throughout the rest of the apartment, but BFS was able to find it right away. I can infer that the closer the three hot dogs and tongs were to the start, the more likely it was that the algorithm felt the need to explore the whole apartment. BFS failed in that it found the hot dogs just after exploring all the tongs, so it looked elsewhere before checking back. UCS failed in that it didn't find all of the hot dogs fast enough, and had to come back to them toward the end in this situation. A* would likely outperform both because of the general knowledge of where the hot dogs and tongs would be—it wouldn't be checking poor locations for them even remotely to the same extent.